

Inside KilimoOrbit Sentinel

What changed during the hardening pass, why each change was made, and how the whole system fits together. It's written so you can defend every decision in front of an investor or a senior engineer.

files changed **21** automated checks **76**, all passing review loops **3** bugs fixed **~35**

THE BRIEF

What "world class" meant for this product

You asked for a world-class application that is ready for marketing and funding. For a tool that tells a farmer where to sell a harvest, that has a concrete meaning. Good looks were never the bar. The bar is five promises the system must keep, and every change in this pass serves one of them.

Never wrong in front of a farmer

A made-up price or a Long Rains warning in January costs someone real money. Bad input must be rejected, and model mistakes must be caught before they reach a phone.

Never down during a demo

Gemini's free tier runs out and sometimes overloads. That happened during this very session. The product has to keep answering anyway.

Provable, not claimed

"It works" is a claim. Seventy-six checks that anyone can run with one command are proof, and investors and accelerator reviewers ask for exactly that.

Safe to put on the internet

An open email endpoint without limits becomes a spam relay. A marketplace where anyone can cancel anyone's listing can be emptied by one script.

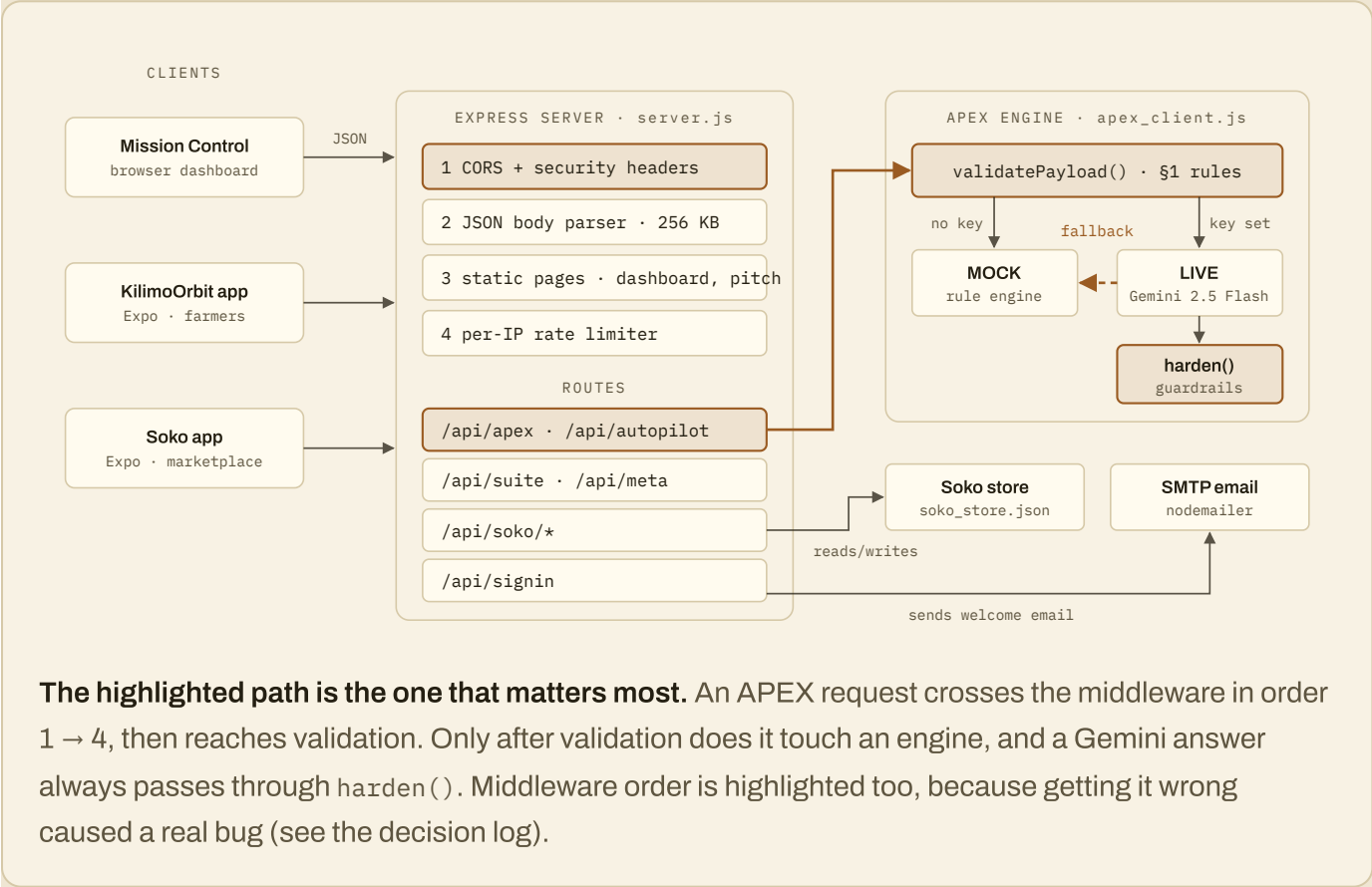
Deployable and presentable

A Docker image, a CI pipeline, a health endpoint, an investor page and a funding calendar. That's the gap between a prototype and a pilot.

SYSTEM DESIGN

The system map

Sentinel is one Node.js process. Three clients talk to it over HTTP with JSON. Inside, requests pass through an ordered stack of middleware before they reach a route. The APEX routes hand the payload to the engine, which picks one of two brains: a deterministic rule engine that runs anywhere, or Gemini. The marketplace and sign-in routes talk to a JSON file and an email server instead.



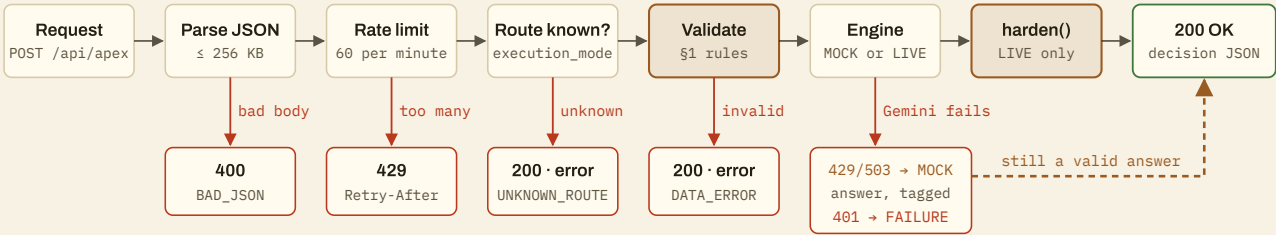
PART	FILE	JOB, IN ONE SENTENCE
System prompt	src/apex_system_prompt.md	The contract: the rules, the five output shapes, the Kenyan climate calendar. Gemini reads it verbatim.
Engine	src/apex_client.js	Validates input, runs MOCK or LIVE, hardens LIVE answers, falls back when Gemini fails.

PART	FILE	JOB, IN ONE SENTENCE
Server	src/server.js	HTTP layer: middleware, rate limits, routes, the Autopilot chain, the telemetry simulator.
Check catalogue	src/suite.js	The 30 contract checks, shared by the command line and the dashboard so they can never disagree.
Marketplace	src/soko/	Listings, claims, deliveries and fair prices, stored in one JSON file.
Dashboard	public/index.html	Mission Control. One file with no build step.
Investor page	public/pitch.html	The one-pager served at /pitch .

SYSTEM DESIGN

One request, start to finish

The best way to understand a backend is to follow one request. Here is a farmer's arbitrage question, `POST /api/apex` . It passes seven gates, and each gate has exactly one way to leave early. The cheap checks come first, so a bad request is turned away before it costs anything.



Red exits end the request early. The dashed amber path is the fallback: the farmer still gets an answer.

Notice that UNKNOWN_ROUTE and DATA_ERROR return HTTP 200. The request reached the engine and the engine answered correctly: "this payload breaks rule §1". That's a business answer, so it travels as a normal response with `execution_mode: "error"`. HTTP error codes are kept for transport problems, meaning a broken body, too many requests, or a missing route.

This is the core of `callApex` , trimmed to its shape. Read it as the diagram written in code:

```
// src/apex_client.js - callApex, simplified
if (!VALID_ROUTES.includes(mode)) return unknownRoute(mode); // gate 4
const { failed, oob } = validatePayload(p, mode); // gate 5
if (failed.length || oob.length) return dataError(failed, oob);

if (engineMode() === "MOCK") return mockEngine(p); // no key: rules only
try {
  return harden(await gemini(p), p); // gates 6 + 7
} catch (err) {
  if (retryable(err)) // 429, 5xx, timeout,
    return { ...mockEngine(p), engine_fallback: { from: "LIVE", to: "MOCK" } };
  throw err; // 401 bad key: never
}
```

WHY EACH CHANGE

Decision log

Engineering teams record important choices as short entries: what was broken, what changed, and the general lesson. These are the ten decisions that matter most. The lesson line in each entry is the part to remember, because it applies to every future project.

Validate in code, not in the prompt

apex_client.js · validatePayload()

BROKEN

Validation only checked that keys existed. A route list sent as a string, onboarding step 9, or a market with no price crashed the engine or produced NaN shillings. In LIVE mode those payloads would also be sent to Gemini and billed.

CHANGED

The engine now checks structure before any engine runs: types, allowed values, array shapes, month names in English or Kiswahili. Each failure names the exact field, down to available_markets[1].wholesale_price_per_kg.

LESSON

Anything that can be checked with an if-statement should never be left to a language model. Code is free, instant and always gives the same answer.

Check the model's answer before a farmer sees it

`apex_client.js` · `harden()`

BROKEN

Gemini output went straight to the client. The prompt forbids inventing prices and requires messages under 25 words, but a model can still slip. In LIVE testing it asked for a yield figure the rules say it should have derived.

CHANGED

`harden()` re-applies the rules that code can check. It hides profit when prices are stale, re-reads the price from the payload, and recomputes when a market name isn't in the data. It also shortens long messages while keeping the emoji, and replaces fake alert IDs. Each intervention is recorded in `guardrail_notes` .

LESSON

Treat model output as untrusted input. Let the model do what only it can do, the language, and let code verify the facts.

Keep answering when Gemini is down

`apex_client.js` · `callApex()` · `retryable()`

BROKEN

A quota error or an overloaded model returned `CLIENT_FAILURE` . Both happened during this session's LIVE test run. In a demo, that's a dead screen.

CHANGED

Temporary failures (429, 5xx, timeouts, network drops) now fall back to the rule engine, and the answer is tagged `engine_fallback` so operators can see what happened. Permanent failures like a wrong API key are deliberately not hidden. The retry delay is also longer after a 429.

LESSON

Degrade gracefully, but only for errors that fix themselves. Hiding a configuration error just moves the outage to a moment when you can't see it.

One list of checks, used everywhere

suite.js · GET /api/suite

BROKEN

The same tests were written three times: in the command-line runner, in the server, and as seven hardcoded names in the dashboard. The server ran 11, so the dashboard silently hid four results.

CHANGED

All 30 checks now live in one module. The command line imports it, the server runs it, and the dashboard fetches the list from GET /api/suite and draws whatever it receives.

LESSON

Keep one source of truth. When the same fact is written in two places, the copies eventually disagree, and nobody notices which one is wrong.

Never invent the headline number

apex_client.js · Route A · yield_basis

BROKEN

Profit is price × yield – transport. The yield came from a hardcoded table, and a missing farm size silently defaulted to one acre. The biggest number on the demo was partly a guess labelled "HIGH confidence".

CHANGED

If the payload includes estimated_yield_kg , it's used exactly as given. Otherwise the yield is derived from the farm size and marked assumed_from_field_area , with a plain-language note. With neither, the request is rejected.

LESSON

Every number you show should be able to say where it came from. Trust with farmers and investors is lost the first time an assumption turns out to be dressed up as data.

Middleware order is behaviour

```
server.js · createApp()
```

BROKEN

The regression review caught this one. The JSON parser ran before the CORS middleware. When a body was malformed, Express jumped straight to the error handler and skipped CORS, so the error response had no `Access-Control-Allow-Origin` header. The browser then hid the real message behind "Failed to fetch".

CHANGED

CORS now runs first, and a test asserts that the header is present on a 400 response. See the diagram below.

LESSON

In Express, the order of `app.use` calls is the program. Headers that every response needs belong at the top, before anything that can fail.

Rate-limit anything that costs money or sends email

```
server.js · rateLimit()
```

BROKEN

The server allows requests from any website. With no limits, any page on the internet could make it send unlimited email or burn the Gemini quota.

CHANGED

A small per-IP limiter with no new dependencies. Sign-in allows 5 per hour, APEX 60 per minute, Autopilot 20, the suite 6 and the marketplace 120. Over the limit, clients get a 429 with a `Retry-After` header. `TRUST_PROXY=1` makes it see real client IPs behind a hosting proxy.

LESSON

Put the tightest limit on the most expensive action. Behind a proxy, check that the limiter sees each user's real IP, or it will lock everyone out together.

Only the owner can withdraw a listing

soko/store.js · owner_token

BROKEN

Any visitor could cancel any farmer's listing. All errors were 400, so the apps couldn't distinguish "doesn't exist" from "not allowed".

CHANGED

Creating a listing returns a secret `owner_token` once, and it is removed from every later read. Cancelling requires it, and the check denies by default. Status codes now mean something: 404 for not found, 403 for not yours, 400 for bad input.

LESSON

Security checks should deny by default. The first version skipped the check for listings with no token, and the reviewer spotted that every older listing was still open.

A crash mid-write must not destroy the data

soko/store.js · write() · read()

BROKEN

Writing straight to the store file meant a crash halfway through could leave half a file. The next read would find broken JSON, start empty, and overwrite every listing.

CHANGED

Each write goes to a temporary file first, then an atomic rename swaps it in. A file that can't be parsed is moved aside and logged, never silently replaced.

LESSON

Write, then rename, is the standard way to update a file safely. A rename either happens completely or not at all.

User text can reach JavaScript internals

apex_client.js · has()

BROKEN

Looking up `YIELD_TABLE[crop]` with the crop `"constructor"` returned a built-in JavaScript function instead of a number. That leaked `NaN` into a farmer-facing sentence. The same trick leaked text into fields that should only hold fixed values.

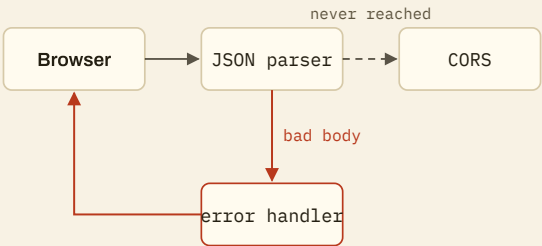
CHANGED

Every lookup keyed by user input now uses `Object.hasOwn()`, and fields with fixed allowed values are checked against an allow-list.

LESSON

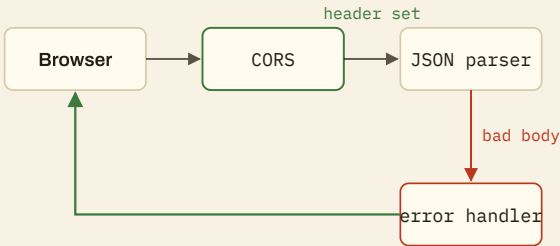
A plain JavaScript object isn't a safe dictionary for untrusted keys. Use `Object.hasOwn` or a `Map`.

BEFORE



!00 without CORS header · browser: "Failed to fetch"

AFTER



400 with CORS header · app shows the real message

Two lines swapped, one bug gone. The fix changed no logic. It only moved CORS above the body parser. Errors that skip middleware are easy to miss because the happy path works perfectly, which is why the HTTP test now sends a deliberately broken body.

PROOF

Four suites, four kinds of confidence

Each suite answers a different question, and none of them can replace another. The rule-engine suite proves the rules are right. The HTTP suite proves the wiring is right, which the rule suite can't see: the CORS bug above passed every rule check. `npm test` runs all four, and CI runs them on every push.

SUITE	CHECKS	QUESTION IT ANSWERS	HOW IT RUNS
APEX contract	30	Does the engine follow every rule in the system prompt, including the odd cases?	Calls <code>callApex</code> directly. Works against MOCK or LIVE.
Soko store	12	Do listings move correctly through open → claimed → delivered, with fair prices?	Calls store functions directly. No server, no model.
HTTP integration	27	Do real requests get the right status codes, headers, limits and response shapes?	Starts the real server on a random port with a temporary data file.
Fallback	7	When Gemini fails, does the right thing happen for each kind of error?	Uses <code>APEX_SIMULATE_LIVE_ERROR</code> to fake 429, 503, timeout and 401.

Every bug in the decision log has a check that fails if it comes back. Case 28 feeds the crop "constructor", and case 27 feeds the month "Bananas". That's how a fix becomes permanent instead of something someone has to remember.

METHOD

How the review loops ran

You asked for repeated loops. The point of a loop is that each pass uses a fresh reader who didn't write the code, because authors are blind to their own assumptions. This was the real sequence:

- 1 Read everything, run the baseline**
All source files, the 392-line system prompt and both mobile clients. 11/11 and 12/12 tests passed, which showed the tests weren't looking hard enough.
- 2 First rewrite**
Engine, server, shared checks, marketplace, dashboard. The check count grew from 11 to 25.
- 3 Independent bug hunt by a separate Opus agent**
It was given read-only access and a list of known issues to skip. It confirmed 22 findings by actually running attack inputs, such as prototype keys, Kiswahili months and huge messages.
- 4 LIVE test against real Gemini**
21 of 25 passed before the quota ran out. The failures showed where the model drifts, which led to the yield rule, the intent guardrail and the fallback.
- 5 Second rewrite**
Fixed the confirmed findings and added a regression check for each. Checks grew to 30, plus the fallback suite.
- 6 Regression review by a third reader**
It reviewed only the diff and asked what the changes broke. It found the CORS ordering bug, the owner-token bypass and the Node 18 incompatibility.
- 7 Third fix pass, screenshots, typechecks**
Headless browser screenshots at desktop, tablet and phone widths caught a layout bug that no test could see. Both Expo apps typecheck cleanly.

What this design doesn't handle yet

Knowing a system's limits is part of understanding it. An investor who asks "what breaks at scale?" should hear these from you first.

One server process only

The store reads, changes and writes the file with synchronous calls, so within one Node process two requests can never interleave. Run two copies of the server and they would overwrite each other. The rate limiter also lives in memory. Move to SQLite or Postgres before scaling out.

The data is simulated

Prices and sensor readings come from a random walk over sample files. The next real milestone is a live price feed (for example KAMIS) and actual sensors.

Yields are regional estimates

When a farm doesn't report its yield, the engine uses conservative averages and says so. Pilot data should replace them.

LIVE isn't fully verified

Gemini passed 21 of 25 before the free quota ran out. The fixes for the failures are covered by rule checks, but the full 30 haven't been run against the live model. Use a paid key or spread the checks out.

No user accounts

The owner token is a lightweight secret, not a login. Anyone holding it can cancel that listing. Real accounts, with phone-number sign-in for example, come before handling money.

The Soko app can't cancel yet

The API supports cancelling. The mobile app doesn't store the owner token or show a cancel button.

Learn it by breaking it

Reading teaches the shape of a system. Breaking it on purpose teaches how it actually works. Run these in Git Bash from `kilimoorbit-sentinel/`.

Suggested reading order

1

`payloads/arbitrage_payload.json`

What a real request looks like. Everything else is about this object.

2

`src/apex_system_prompt.md` , sections 1 and 3

The rules. The code exists to enforce them.

3

`callApex` at the bottom of `src/apex_client.js`

The request journey in about 40 lines. Then `validatePayload`, Route A in `mockEngine`, and `harden`.

4

`createApp` in `src/server.js`

Read the middleware from top to bottom and match it to the system map.

5

`src/tests/server_tests.js`

Every endpoint, called the way the apps call it.

Five experiments

```
# 1 · See all four suites pass
APEX MOCK=1 npm test
```

```
# 2 · Reproduce the CORS bug: move the express.json line in src/server.js
#     above the CORS block, then watch exactly one check fail
npm run test:server
```

```
# 3 · Pretend Gemini is overloaded, and look for "engine_fallback" in the output
GEMINI_API_KEY=fake APEX_SIMULATE_LIVE_ERROR=503 npm run route:arbitrage
```

```
# 4 · Pretend the key is wrong: this should FAIL loudly, not fall back
GEMINI_API_KEY=fake APEX_SIMULATE_LIVE_ERROR=401 npm run route:arbitrage
```

```
# 5 · Hit the sign-in limit (start the server first with: npm start)
for i in 1 2 3 4 5 6; do
  curl -s -X POST localhost:4517/api/signin -H 'Content-Type: application/json' \
    -d '{"name":"Test","email":"t@example.com"}'; echo
done
```

For a sixth experiment, write your own rule. Add a check to `harden()` for Route E: the revised profit should never be larger than the original. Then add a matching case to `src/suite.js`. Doing that once teaches you the full cycle of rule, guardrail and proof.

TERMS

Glossary

Contract

The agreed shape of inputs and outputs. Here, the system prompt defines it and the checks prove it.

Deterministic

Same input, same output, every time. The rule engine is deterministic. A language model isn't, even at temperature 0.

Guardrail

Code that inspects a model's answer and corrects or blocks what breaks the rules.

Graceful degradation

Keep working at reduced quality when a dependency fails, instead of failing completely.

Middleware

A function every request passes through in order before reaching its route. Order changes behaviour.

CORS

Browser rules on which websites may read a server's responses. The server permits it with response headers.

Rate limiting

Capping requests per client per time window, to protect cost and stop abuse.

Atomic write

An update that either happens completely or not at all. Here: write a temporary file, then rename it.

Prototype pollution

When user-supplied keys like `constructor` reach JavaScript's built-in object properties.

Regression check

A test written for a fixed bug, so that bug can't quietly return.

Fixture

A saved sample input, like the files in `payloads/`, used to make tests repeatable.